Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación Campus Portales

Ingeniería en Sistemas de Información y Ciencias de la Computación

Curso:

Análisis de Sistemas II

Catedrático:

Antonio

Proyecto Final Fase 3: Módulo de ventas

Integrantes:	Carné
Dailin Mireya Xepuxtián Hernández	9989-23-1403
Kevin Esteban Quinteros Quiñonez	9989-23-17397
Sofia Abigail Morales Pineda	9989-22-9952

Sección: “A”

Semestre II – Ciclo 8

INDICE

1. INTRODUCCIÓN Y RESUMEN EJECUTIVO 3

1.1 Propósito del proyecto 3

1.2 Alcance del software 3

1.3 Resumen ejecutivo 3

1.4 Síntesis de los 10 pasos metodológicos 4

2. REQUERIMIENTOS, MODELADO UML 2.5.1 Y GESTIÓN ÁGIL 5

2.1 Perspectiva del producto 5

2.2 Tipos de usuario 5

2.3 Especificación de Requerimientos del Sistema (ISO/IEC/IEEE 29148:2018) 6

2.4 Diagramas UML 2.5.1 12

2.5 Fichas de Diseño de Componentes - SDD (IEEE 1016-2009) 13

2.5.1. Ficha de diseño del módulo Autenticación y seguridad 13

2.5.2. Ficha de diseño del módulo Catálogo de productos 15

2.5.3. Ficha de diseño del módulo Inventario 17

2.5.4. Ficha de diseño del módulo Gestión de Cotizaciones 19

2.5.5. Ficha de diseño del módulo Gestión de clientes 21

2.5.6. Ficha de diseño del módulo Gestión de proveedores 23

2.5.7. Ficha de diseño del módulo Pagos 25

2.5.8. Ficha de diseño del módulo Envíos 27

2.4 Gestión Ágil de Proyectos 29

2.4.1 Marco de trabajo (Scrum/Kanban) y rol del Product Owner. 29

2.4.2 Product Backlog con Historias de Usuario (INVEST / 3C) 30

3. EVALUACIÓN ARQUITECTÓNICA, C4 Y PERSISTENCIA 33

3.1 Evaluación de Estilos Arquitectónicos y Escenarios de Calidad (ISO/IEC 25010) 33

3.2 Registros de Decisiones de Arquitectura (ADR - ISO/IEC/IEEE 42010:2011) 35

3.3 Documentación de la Arquitectura mediante el Modelo C4 (4 Niveles) 41

3.4 Arquitectura de Persistencia Políglota (PostgreSQL, MongoDB, Redis, Neo4j) y Cache-Aside 44

3.5 Aplicación de Patrones GoF y Análisis de Cohesión / Acoplamiento 45

1. INTRODUCCIÓN Y RESUMEN EJECUTIVO

1.1 Propósito del proyecto

El proyecto tiene como propósito analizar, especificar y diseñar el módulo de Ventas y cotizaciones de Giganet, S.A., un sistema de comercio electrónico especializado en dispositivos de red. El sistema permite al cliente consultar productos, armar su carrito, registrar sus datos de entrega, elegir un método de pago y confirmar su pedido. También permite al administrador dar seguimiento a las ventas realizadas.

Además del producto, el proyecto busca aplicar una metodología completa de ingeniería de software, desde la definición de requisitos hasta el diseño modular en capas. Esto se hace con estándares reconocidos (ISO/IEC/IEEE 29148, UML 2.5.1 e IEEE 1016-2009) y con prácticas ágiles (Product Backlog, historias de usuario y criterios de calidad).

1.2 Alcance del software

Incluye:

- Módulos del sistema: Autenticación y seguridad, Catálogo de productos, Inventario, Gestión de cotizaciones(orden de compra), Gestión de clientes, Gestión de proveedores, Pagos y Envíos.
- Flujo de venta de principio a fin: consulta, búsqueda y filtrado de productos, administración del carrito, resumen de compra, datos de entrega, método de pago, confirmación y número de pedido.
- Consultas de seguimiento: estado del pedido e historial de compras para el cliente, y ventas realizadas para el administrador.
- Reglas de negocio: 14 reglas (RN-01 a RN-14) que regulan existencias, precios, validaciones, pagos y permisos de consulta.
- Backlog: 15 historias de usuario en 6 épicas, de las cuales 12 forman parte del Producto Mínimo Viable (MVP).

No incluye:

- Cobros reales: la pasarela de pago es solo una simulación.

1.3 Resumen ejecutivo

Se partió de la necesidad de ordenar el proceso de venta de dispositivos de red en un sistema modular, mantenible y verificable. A partir de los objetivos del negocio se definió el rol del Analista / Product Owner, se construyó un Product Backlog priorizado con MoSCoW y Story Points, y se documentó cada historia con el formato 3C, validada con INVEST y controlada por una Definition of Ready y una Definition of Done.

Los requisitos se formalizaron en una Especificación de Requisitos (SRS) bajo ISO/IEC/IEEE 29148, con requerimientos funcionales y no funcionales verificables y una matriz de trazabilidad. Después se modelaron los diagramas UML 2.5.1 (clases de análisis y diseño, secuencia, actividades con calles y paquetes) y se documentó cada módulo con una ficha de diseño según IEEE 1016-2009. Cada ficha detalla atributos, interfaces, dependencias y manejo de excepciones.

El resultado es un conjunto de artefactos coherentes y trazables, desde la necesidad del negocio hasta el diseño de cada módulo, listo para la implementación.

1.4 Síntesis de los 10 pasos metodológicos

Paso	Actividad	Resultado
1	Definición del contexto, objetivo y alcance del módulo de Ventas	Objetivo, épicas y módulos del sistema
2	Definición del rol del Analista / Product Owner	Responsabilidades, autoridad y entregables del rol
3	Construcción del Product Backlog en GitHub Projects	Backlog priorizado (MoSCoW, Story Points, MVP)
4	Historias de usuario en formato 3C, validación INVEST, DoR y DoD	15 historias con Card, Conversation y Confirmation
5	Definición de reglas de negocio	RN-01 a RN-14 y su relación con las historias
6	Especificación de requisitos (ISO/IEC/IEEE 29148)	SRS con REQ-FN y REQ-NF verificables
7	Trazabilidad de requisitos	Matriz historia, requerimiento y prioridad
8	Modelado UML 2.5.1	Diagramas de clases, secuencia, actividades y paquetes
9	Diseño modular según IEEE 1016-2009	Fichas de diseño por módulo
10	Revisión y consolidación de los artefactos	Documento final del sistema

2. REQUERIMIENTOS, MODELADO UML 2.5.1 Y GESTIÓN ÁGIL

2.1 Perspectiva del producto

Se organiza en las siguientes épicas del Product Backlog(trabajado en la fase anterior):

- EP-01 Catálogo y disponibilidad: consultar los dispositivos de red disponibles para la venta.
- EP-02 Carrito de compras: seleccionar productos y administrar cantidades.
- EP-03 Gestión del proceso de venta: registrar y confirmar una venta.
- EP-04 Pagos: gestionar la información y el estado del pago.
- EP-05 Gestión del pedido: consultar y actualizar el estado de las órdenes.
- EP-06 Historial y seguimiento: permitir al cliente consultar sus compras anteriores y su estado.
- EP-07 Usuarios y autenticación: permitir el registro, el inicio y el cierre de sesión.

2.2 Tipos de usuario

- Cliente: se registra, inicia sesión, consulta el catálogo, arma su carrito, realiza la compra y da seguimiento a sus pedidos.
- Administrador: inicia sesión, consulta las ventas realizadas y actualiza el estado de los pedidos.

2.3 Especificación de Requerimientos del Sistema (ISO/IEC/IEEE 29148:2018)

2.3.1. Requerimientos funcionales

ID	Enunciado	Prioridad	Fuente	Verificación	Criterio de aceptación
REQ-FN-001	Cuando el cliente accede a la sección de catálogo, el sistema deberá mostrar los dispositivos de red con nombre, código, precio y disponibilidad en un máximo de 3 s.	Debe	US-01	Prueba	Se listan todos los productos activos con los cuatro datos visibles y la carga tarda 3 s o menos.
REQ-FN-002	Cuando el cliente ingresa un nombre o código en el buscador, el sistema deberá mostrar los productos coincidentes, incluso con coincidencia parcial, en un máximo de 3 s.	Debe	US-02	Prueba	Al buscar «router» se listan todos los productos que lo contengan en nombre o código; sin coincidencias se muestra el mensaje «Sin resultados».
REQ-FN-003	Cuando el cliente selecciona filtros de categoría, marca o rango de precio, el sistema deberá mostrar únicamente los productos que cumplan todos los filtros activos.	Debería	US-03	Prueba	Con categoría y marca combinadas no aparece ningún producto que incumpla alguno de los dos filtros.
REQ-FN-004	Cuando el cliente selecciona un producto, el sistema deberá mostrar sus características, precio y disponibilidad en la página de detalle.	Debe	US-04	Demostración	El detalle muestra los tres datos y coinciden con los registrados del producto.
REQ-FN-005	Cuando el cliente solicita agregar un producto con existencia disponible, el sistema deberá agregarlo al carrito y actualizar el contador del carrito en un máximo de 1 s.	Debe	US-05	Prueba	El producto aparece en el carrito; si no hay existencia, se bloquea la acción y se muestra un aviso.
REQ-FN-006	Cuando el cliente cambia la cantidad de un producto en el carrito, el sistema deberá recalcular subtotal y total en un máximo de 1 s, sin permitir una cantidad mayor a la existencia.	Debe	US-06	Prueba	Con cantidad válida el total se actualiza correctamente; con cantidad mayor a la existencia se rechaza el cambio.

ID	Enunciado	Prioridad	Fuente	Verificación	Criterio de aceptación
REQ-FN-007	Cuando el cliente elimina un producto del carrito antes de confirmar la compra, el sistema deberá quitarlo y recalcular el total en un máximo de 1 s.	Debe	US-07	Prueba	El producto desaparece del carrito y el total es igual a la suma de los productos restantes.
REQ-FN-008	Cuando el cliente inicia la confirmación de la compra, el sistema deberá mostrar el resumen con productos, cantidades, subtotal y total.	Debe	US-08	Prueba	El total del resumen es igual a la suma de los subtotales del carrito.
REQ-FN-009	Cuando el cliente continúa con la compra, el sistema deberá registrar nombre del receptor, dirección y teléfono de entrega, rechazando el envío si falta algún campo obligatorio.	Debe	US-09	Prueba	Con los campos completos los datos se guardan; con algún campo vacío se muestra un error y no se avanza al pago.
REQ-FN-010	Cuando el cliente llega al paso de pago, el sistema deberá permitirle seleccionar un método de pago de los habilitados antes de completar la compra.	Debe	US-10	Demostración	No es posible finalizar la compra sin haber seleccionado un método de pago.
REQ-FN-011	Cuando la transacción de pago es procesada, el sistema deberá mostrar al cliente la confirmación con su resultado (aprobado o rechazado) en un máximo de 5 s.	Debería	US-11	Prueba	Tras el pago se muestra el resultado correcto de la transacción en 5 s o menos.
REQ-FN-012	Cuando la compra es confirmada, el sistema deberá generar un número de pedido único y mostrarlo al cliente.	Debe	US-12	Prueba	No existen dos pedidos(Cotizaciones de compra) con el mismo número y este se muestra al terminar la compra.
REQ-FN-013	Cuando el cliente consulta un pedido, el sistema deberá mostrar su estado actual.	Debería	US-13	Demostración	El estado mostrado coincide con el último estado registrado para ese pedido.
REQ-FN-014	Cuando el cliente autenticado accede a su historial, el sistema deberá listar sus compras anteriores	Podría	US-14	Prueba	Solo aparecen compras del cliente autenticado

ID	Enunciado	Prioridad	Fuente	Verificación	Criterio de aceptación
REQ-FN-015	Cuando el administrador accede al módulo de ventas, el sistema deberá listar las ventas realizadas con fecha, cliente y total.	Debe	US-15	Prueba	Cada venta registrada aparece una sola vez con sus tres datos.
REQ-FN-016	Cuando el administrador cambia el estado de un pedido, el sistema deberá guardar el nuevo estado y mostrarlo al cliente en su siguiente consulta.	Debería	US-16	Prueba	Tras el cambio, el cliente ve el nuevo estado al consultar su pedido.
REQ-FN-017	Cuando una persona completa el formulario de registro con nombre, correo electrónico y contraseña válidos, el sistema deberá crear una cuenta de cliente en un máximo de 3 s.	Debe	US-17*	Prueba	La cuenta queda creada y la persona puede iniciar sesión con ella; con un campo obligatorio vacío o inválido el registro se rechaza.
REQ-FN-018	Cuando una persona intenta registrarse con un correo electrónico ya registrado, el sistema deberá rechazar el registro e informar que el correo ya existe.	Debe	US-17*	Prueba	No se crea una segunda cuenta con el mismo correo y se muestra el mensaje de correo existente.
REQ-FN-019	Cuando un usuario ingresa credenciales válidas, el sistema deberá iniciar su sesión y redirigirlo a la vista correspondiente a su rol (cliente o administrador) en un máximo de 3 s.	Debe	US-18*	Prueba	Un cliente accede a la tienda y un administrador accede al módulo de administración.
REQ-FN-020	Cuando un usuario ingresa credenciales inválidas, el sistema deberá denegar el acceso y mostrar un mensaje que no indique cuál de los datos es incorrecto.	Debe	US-18*	Prueba	No se inicia sesión y el mensaje es el mismo si el correo no existe o la contraseña es incorrecta.
REQ-FN-021	Cuando el usuario selecciona cerrar sesión, el sistema deberá finalizar su sesión y redirigirlo a la página de inicio en un máximo de 1 s.	Debería	US-19*	Demostración	Después de cerrar sesión no se puede acceder a páginas protegidas sin volver a iniciar sesión.

* Usuarios y autenticación.

2.3.2. Requerimientos funcionales

Se establecen las cualidades que debe cumplir el sistema. En lugar de una historia de usuario, la columna *Fuente* indica la categoría de calidad.

ID	Enunciado	Prioridad	Categoría	Verificación	Criterio de aceptación
REQ-NF-001	Cuando hay hasta 100 usuarios concurrentes, el sistema deberá responder a las consultas de catálogo y búsqueda en un máximo de 3 s.	Debe	Rendimiento	Prueba	En una prueba de carga con 100 usuarios simulados, el 95 % de las respuestas tarda 3 s o menos.
REQ-NF-002	Cuando se almacena una contraseña, el sistema deberá guardarla cifrada mediante un algoritmo de hash seguro (por ejemplo, bcrypt).	Debe	Seguridad	Inspección	Ninguna contraseña aparece en texto plano en la base de datos.
REQ-NF-003	Cuando el cliente se comunica con el servidor, el sistema deberá transmitir toda la información mediante HTTPS.	Debe	Seguridad	Inspección	No es posible acceder al sistema por HTTP sin cifrar.
REQ-NF-004	Cuando un usuario sin rol de administrador intenta acceder a las ventas o a la actualización de pedidos, el sistema deberá denegar el acceso.	Debe	Seguridad	Prueba	Un cliente que intenta abrir estas funciones recibe un mensaje de acceso denegado.
REQ-NF-005	Cuando el cliente accede desde un equipo de escritorio, una tableta o un móvil, el sistema deberá adaptar su interfaz al tamaño de la pantalla.	Debe	Usabilidad	Demostración	No hay desplazamiento horizontal en anchos de 360 px, 768 px y 1366 px.
REQ-NF-006	Cuando un cliente busca un producto, el sistema deberá permitirle agregarlo al carrito en un máximo de 5 clics.	Debería	Usabilidad	Prueba	Desde la página de inicio, un usuario nuevo completa la tarea en 5 clics o menos.
REQ-NF-007	El sistema deberá estar disponible al menos el 99 % del mes.	Debería	Disponibilidad	Análisis	Monitoreo mensual de 99 % o más (máximo aproximado de 7 h de caída).
REQ-NF-008	Cuando el cliente accede desde las dos últimas versiones de Chrome, Firefox, Edge o Safari, el sistema deberá funcionar sin errores.	Debe	Compatibilidad	Prueba	El flujo completo de compra se realiza sin errores en los cuatro navegadores.

ID	Enunciado	Prioridad	Categoría	Verificación	Criterio de aceptación
REQ-NF-009	Cuando la sesión del cliente se interrumpe temporalmente, el sistema deberá conservar el contenido de su carrito durante 24 h.	Podría	Fiabilidad	Prueba	Al volver a iniciar sesión dentro de 24 h, el carrito contiene los mismos productos y cantidades.
REQ-NF-010	El sistema deberá estar organizado en módulos independientes (catálogo, carrito, ventas, pagos, pedidos y usuarios) para facilitar su mantenimiento.	Debería	Mantenibilidad	Inspección	El código fuente está separado por módulos y cada uno se puede modificar sin cambiar a los demás.
REQ-NF-011	Cuando un usuario falla el inicio de sesión 5 veces consecutivas, el sistema deberá bloquear temporalmente el acceso a esa cuenta durante 15 min.	Debería	Seguridad	Prueba	Tras el quinto intento fallido se rechaza incluso una contraseña correcta hasta que pasen 15 min.
REQ-NF-012	Cuando una sesión permanece inactiva durante 30 min, el sistema deberá cerrarla automáticamente.	Debería	Seguridad	Prueba	Al intentar una acción después de 30 min de inactividad se solicita iniciar sesión de nuevo.
REQ-NF-013	Cuando una persona crea o cambia su contraseña, el sistema deberá exigir una longitud mínima de 8 caracteres que incluya letras y números.	Debe	Seguridad	Prueba	Una contraseña que incumple la regla es rechazada con un mensaje que explica el requisito.

2.3.3. Matriz de trazabilidad

Cada requerimiento funcional se origina en una historia de usuario del Product Backlog(Tablero Git).

Historia	Épica	Requerimiento	MoSCoW
US-01	EP-01 Catálogo y disponibilidad	REQ-FN-001	Must
US-02	EP-01 Catálogo y disponibilidad	REQ-FN-002	Must
US-03	EP-01 Catálogo y disponibilidad	REQ-FN-003	Should
US-04	EP-01 Catálogo y disponibilidad	REQ-FN-004	Must
US-05	EP-01 Catálogo y disponibilidad	REQ-FN-005	Must
US-06	EP-02 Carrito de compras	REQ-FN-006	Must
US-07	EP-02 Carrito de compras	REQ-FN-007	Must
US-08	EP-03 Gestión del proceso de venta	REQ-FN-008	Must
US-09	EP-03 Gestión del proceso de venta	REQ-FN-009	Must
US-10	EP-04 Pagos	REQ-FN-010	Must
US-11	EP-04 Pagos	REQ-FN-011	Should
US-12	EP-05 Gestión del pedido	REQ-FN-012	Must
US-13	EP-05 Gestión del pedido	REQ-FN-013	Should
US-14	EP-06 Historial y seguimiento	REQ-FN-014	Could
US-15	EP-03 Gestión del proceso de venta	REQ-FN-015	Must
US-16	EP-05 Gestión del pedido	REQ-FN-016	Should
US-17*	EP-07 Usuarios y autenticación*	REQ-FN-017, REQ-FN-018	Must
US-18*	EP-07 Usuarios y autenticación*	REQ-FN-019, REQ-FN-020	Must
US-19*	EP-07 Usuarios y autenticación*	REQ-FN-021	Should

* Historias y épicas propuestas en fase 2.

2.4 Diagramas UML 2.5.1

Diagrama / Módulo	Imagen (PNG / Visualización)	Archivo Fuente (UML / Draw.io / MMD)
0. Contenedor General	—	Mi-proyecto-sistema2/docs/uml/0_diagramas_uml.drawio
1. Diagrama de Clases	Mi-proyecto-sistema2/docs/uml/1_clases.png.png	Mi-proyecto-sistema2/docs/uml/1_clases.puml
2.1. Secuencia (Pago y Envío)	Mi-proyecto-sistema2/docs/uml/2.1_secuencia_pago-envio.drawio	Mi-proyecto-sistema2/docs/uml/2.1_secuencia_pago_envio.puml
2.2. Secuencia (Endpoint)	Mi-proyecto-sistema2/docs/uml/2.2_secuencia_endpoint	Mi-proyecto-sistema2/docs/uml/2.2_secuencia_endpoint.mmd
3. Diagrama de Actividades	Mi-proyecto-sistema2/docs/uml/3_actividades	Mi-proyecto-sistema2/docs/uml/3_actividades.puml
4. Diagrama de Paquetes	Mi-proyecto-sistema2/docs/uml/4_paquetes	Mi-proyecto-sistema2/docs/uml/4_paquetes.puml

2.5 Fichas de Diseño de Componentes - SDD (IEEE 1016-2009)

2.5.1. Ficha de diseño del módulo Autenticación y seguridad

MOD-AUT v1.0 · IEEE 1016-2009

Código	MOD-AUT v1.0
Responsabilidad	Identificar a los usuarios del sistema, controlar sus permisos por rol y proteger el acceso a los demás módulos mediante sesiones seguras y registro de auditoría.
Estados	Registrado → Activo → Bloqueado → Activo (por recuperación) | Inactivo

1. Atributos

Entidad	Atributo	Tipo	Restricción
Usuario	idUsuario	UUID, PK	Único e inmutable
Usuario	correo	varchar(120)	Único, formato válido
Usuario	hashContrasena	varchar(255)	Bcrypt/Argon2; nunca en texto plano
Usuario	estado	enum	Activo, Bloqueado, Inactivo
Usuario	intentosFallidos	int	Bloqueo al llegar a 5
Rol	idRol / nombre	int PK / varchar(40)	Admin, Cliente, Proveedor, Operador
Sesion	token / expiracion	varchar / timestamp	JWT con vigencia de 30 min
Auditoria	accion / fecha / ip	varchar / timestamp / varchar	Solo inserción (no editable)

2. Interfaces

Provistas	POST /v1/auth/login POST /v1/auth/logout POST /v1/auth/refresh POST /v1/auth/recuperar-contrasena GET /v1/auth/validar-token (uso interno de los módulos) Evento: UsuarioBloqueado
Requeridas	Gestión de clientes (datos de contacto para recuperación)

3. Dependencias

Detalle	Internas: Gestión de clientes, Gestión de proveedores (alta de credenciales). Externas: MySQL (usuarios y roles), Redis (sesiones y lista de tokens revocados), servidor SMTP.

4. Manejo de excepciones

Excepción	Causa	HTTP	Acción del módulo
CredencialesInvalidas	Correo o contraseña incorrectos	401	Incrementa intentos y registra en auditoría
CuentaBloqueada	Cinco intentos fallidos consecutivos	423	Rechaza el acceso y envía código de desbloqueo
TokenExpirado	Sesión vencida	401	Solicita renovación con refresh
AccesoDenegado	El rol no tiene permiso sobre el recurso	403	Rechaza y registra el intento
ServicioNoDisponible	Falla de Redis o de la base de datos	503	Deniega el acceso por seguridad y reintenta

5. Restricciones

- Autenticación en menos de 500 ms (p95).
- Contraseñas cifradas con hash y sal; comunicación solo por HTTPS.
- Cumplimiento de control de acceso basado en roles (RBAC).

6. Vistas de IEEE 1016 usadas

- Composición: diagrama de paquetes.
- Interfaz: contrato OpenAPI.
- Interacción: secuencia del inicio de sesión.
- Información: modelo de datos Usuario–Rol.
- Dinámica de estados: ciclo de vida de la cuenta.

2.5.2. Ficha de diseño del módulo Catálogo de productos

MOD-CAT v1.0 · IEEE 1016-2009

Código	MOD-CAT v1.0
Responsabilidad	Mantener el catálogo de productos (datos, categorías y precios) y ofrecer su consulta a los demás módulos. No gestiona existencias: eso corresponde al módulo Inventario.
Estados	Borrador → Activo → Descontinuado

1. Atributos

Entidad	Atributo	Tipo	Restricción
Producto	idProducto	UUID, PK	Único e inmutable
Producto	sku	varchar(30)	Único, obligatorio
Producto	nombre / descripcion	varchar(150) / texto	Nombre obligatorio
Producto	precio	decimal(10,2)	Mayor que 0
Producto	idCategoria	FK	Debe existir
Producto	idProveedor	FK	Debe existir en Proveedores
Producto	imagenUrl	varchar(255)	Opcional
Producto	estado	enum	Borrador, Activo, Descontinuado
Categoria	idCategoria / nombre	int PK / varchar(60)	Nombre único
HistorialPrecio	idProducto / precioAnterior / fecha	FK / decimal / timestamp	Solo inserción

2. Interfaces

Provistas	GET /v1/productos GET /v1/productos/{id} GET /v1/productos?categoria={id}&q={texto} POST /v1/productos PUT /v1/productos/{id} PATCH /v1/productos/{id}/estado GET /v1/categorias Evento: ProductoActualizado
Requeridas	Autenticación y seguridad (validarToken, permisos de edición) Gestión de proveedores (consultarProveedor) Inventario (disponibilidad para mostrar en la consulta)

3. Dependencias

Detalle	Internas: Autenticación y seguridad, Proveedores, Inventario. Externas: PostgreSQL (catálogo), Redis (caché de consultas), almacenamiento de imágenes, broker de eventos.

4. Manejo de excepciones

Excepción	Causa	HTTP	Acción del módulo
ProductoNoEncontrado	El id o SKU no existe	404	Devuelve error sin modificar datos
SkuDuplicado	Ya existe un producto con ese SKU	409	Rechaza el alta
PrecioInvalido	Precio menor o igual a cero	422	Rechaza la operación
CategoriaInexistente	La categoría indicada no existe	422	Rechaza la operación
AccesoDenegado	El rol no puede editar el catálogo	403	Rechaza y registra el intento
ServicioNoDisponible	Falla de base de datos o caché	503	Consulta directa a la base de datos o error controlado

5. Restricciones

- Consulta de catálogo en menos de 300 ms (p95).
- Solo se muestran productos en estado Activo.
- Todo cambio de precio queda en el historial.

6. Vistas de IEEE 1016 usadas

- Composición: diagrama de paquetes.
- Interfaz: contrato OpenAPI.
- Interacción: secuencia de consulta de producto.
- Información: modelo de datos Producto–Categoría.
- Dinámica de estados: ciclo de vida del producto.

2.5.3. Ficha de diseño del módulo Inventario

MOD-INV v1.0 · IEEE 1016-2009

Código	MOD-INV v1.0
Responsabilidad	Controlar las existencias de cada producto: entradas, salidas, reservas, ajustes y alertas de stock mínimo.
Estados	Stock: Disponible → Reservado → Descontado | Liberado (por cancelación o expiración)

1. Atributos

Entidad	Atributo	Tipo	Restricción
Inventario	idProducto	FK, PK	Un registro por producto
Inventario	cantidadDisponible	int	No negativa
Inventario	cantidadReservada	int	Menor o igual a la disponible
Inventario	stockMinimo	int	Mayor o igual a 0
Inventario	ubicacion	varchar(40)	Opcional
MovimientoInventario	idMovimiento	UUID, PK	Único
MovimientoInventario	tipo	enum	Entrada, Salida, Ajuste, Reserva, Liberación
MovimientoInventario	cantidad / fecha / idReferencia	int / timestamp / varchar	Cantidad distinta de 0; solo inserción
Reserva	idReserva / idOrden / cantidad / expiracion	PK / FK / int / timestamp	Expira a los 15 min si no se confirma

2. Interfaces

Provistas	GET /v1/inventario/{idProducto} POST /v1/inventario/reservar POST /v1/inventario/liberar POST /v1/inventario/confirmar-salida POST /v1/inventario/entradas POST /v1/inventario/ajustes Evento: StockBajo
Requeridas	Catálogo de productos (validar que el producto exista) Gestión de proveedores (registro de entrada por reabastecimiento) Gestión de órdenes de compra (confirmación de salidas) Autenticación y seguridad (validarToken)

3. Dependencias

Detalle	Internas: Catálogo, Proveedores, Órdenes de compra, Autenticación. Externas: PostgreSQL (existencias y movimientos), Redis (caché de consulta), broker de eventos.

4. Manejo de excepciones

Excepción	Causa	HTTP	Acción del módulo
ProductoNoEncontrado	El producto no existe en el catálogo	404	Devuelve error sin cambios
StockInsuficiente	Cantidad solicitada mayor a la disponible	409	Rechaza la reserva e informa lo disponible
ConflictoConcurrencia	Dos reservas simultáneas sobre el mismo stock	409	Bloqueo optimista y reintento
CantidadInvalida	Cantidad igual a 0 o negativa	422	Rechaza la operación
ReservaExpirada	Se confirma una reserva ya vencida	410	Libera stock y solicita nueva reserva
ServicioNoDisponible	Falla de base de datos o caché	503	Error controlado; no se modifica el stock

5. Restricciones

- Reserva de inventario con transacción ACID.
- Las reservas no confirmadas se liberan automáticamente a los 15 min.
- Todo movimiento queda registrado para auditoría.

6. Vistas de IEEE 1016 usadas

- Composición: diagrama de paquetes.
- Interfaz: contrato OpenAPI.
- Interacción: secuencia de reserva de stock.
- Información: modelo de datos Inventario–Movimiento.
- Dinámica de estados: ciclo del stock.

2.5.4. Ficha de diseño del módulo Gestión de Cotizaciones

MOD-ORD v1.0 · Área 1 · IEEE 1016-2009

Código	MOD-ORD v1.0
Responsabilidad	Crear, validar y dar seguimiento a las Cotizaciones desde su registro hasta la entrega, coordinando catálogo, pagos y envío.
Estados	Recibida → Pagada → En preparación → En camino → Entregada (o Cancelada)

1. Atributos

Entidad	Atributo	Tipo	Restricción
Cotización	idCotizaciones	UUID, PK	Único; clave de idempotencia
Cotizaciones	idCliente	FK	Cliente activo
Cotizaciones	fechaCreacion	timestamp	Automática
Cotizaciones	estado	enum	Recibida, Pagada, En preparación, En camino, Entregada, Cancelada
Cotizaciones	total	decimal(10,2)	Suma de subtotales + impuestos
Cotizaciones	direccionEntrega	texto	Obligatoria
DetalleCotizaciones	idDetalle / idCotizaciones	PK / FK	—
DetalleCotizaciones	idProducto	FK	Existente en catálogo
DetalleCotizaciones	cantidad / precioUnitario / subtotal	int / decimal / decimal	Cantidad mayor que 0

2. Interfaces

Provistas	POST /v1/Cotizaciones GET /v1/Cotizaciones/{id} GET /v1/Cotizaciones?cliente={id} PATCH /v1/Cotizaciones/{id}/estado DELETE /v1/Cotizaciones/{id} (cancelación) Evento: CotizacionesCreada Evento: CotizacionesCancelada
Requeridas	Autenticación (validarToken) Gestión de clientes (validarCliente) Catálogo e inventario (consultarPrecio, reservarStock) Pagos y envío (autorizarPago, asignarEnvio)

3. Dependencias

Detalle	Internas: Autenticación, Clientes, Catálogo e inventario, Pagos y envío. Externas: PostgreSQL (órdenes y detalle), Redis (carrito), broker de eventos.

4. Manejo de excepciones

Excepción	Causa	HTTP	Acción del módulo
ClienteNoValido	Cliente inexistente o inactivo	404	Rechaza la Cotizaciones
StockInsuficiente	Catálogo no puede reservar el producto	409	Rechaza la Cotizaciones y notifica al cliente
PagoRechazado	Pagos no autoriza la transacción	402	Cotizaciones queda en Recibida; permite reintento
CotizacionesDuplicada	Mismo idCotizaciones recibido dos veces	200	Devuelve la Cotizaciones existente (idempotencia)
TransicionInvalida	Cambio de estado fuera del ciclo de vida	422	Rechaza el PATCH y registra el intento
ServicioNoDisponible	Timeout de Catálogo, Pagos o Envío	503	Reintento con backoff; si falla, cancela y libera stock

5. Restricciones

- Confirmación en menos de 2 s (p95).
- Transacciones ACID e idempotencia por idCotizaciones.
- Compensación (liberar stock) ante fallo parcial.

6. Vistas de IEEE 1016 usadas

- Composición: diagrama de paquetes.
- Interfaz: contrato OpenAPI.
- Interacción: secuencia de creación de la Cotizaciones.
- Información: modelo de datos Cotizaciones–Detalle.
- Dinámica de estados: ciclo de vida de la Cotizaciones.

2.5.5. Ficha de diseño del módulo Gestión de clientes

MOD-CLI v1.0 · Área 2 · IEEE 1016-2009

Código	MOD-CLI v1.0
Responsabilidad	Registrar y mantener la información de los clientes, sus direcciones y su historial de compras, respetando la protección de datos personales.
Estados	Registrado → Activo → Suspendido → Activo | Inactivo

1. Atributos

Entidad	Atributo	Tipo	Restricción
Cliente	idCliente	UUID, PK	Único
Cliente	nombre / apellido	varchar(80)	Obligatorios
Cliente	correo / telefono	varchar(120) / varchar(20)	Correo único y válido
Cliente	nit / documento	varchar(20)	Opcional; formato validado
Cliente	estado	enum	Activo, Inactivo, Suspendido
Cliente	fechaRegistro	timestamp	Automática
Direccion	idDireccion / idCliente	PK / FK	—
Direccion	calle / ciudad / departamento / codigoPostal	varchar	Al menos una dirección principal
Consentimiento	tipo / aceptado / fecha	enum / bool / timestamp	Registro de privacidad

2. Interfaces

Provistas	POST /v1/clientes GET /v1/clientes/{id} PUT /v1/clientes/{id} GET /v1/clientes/{id}/direcciones POST /v1/clientes/{id}/direcciones GET /v1/clientes/{id}/Cotizaciones Evento: ClienteRegistrado
Requeridas	Autenticación (validarToken, crearCredenciales) Gestión de Cotizaciones (historial de compras)

3. Dependencias

Detalle	Internas: Autenticación, Cotizaciones. Externas: MySQL (clientes y direcciones)

4. Manejo de excepciones

Excepción	Causa	HTTP	Acción del módulo
ClienteNoEncontrado	El id no existe	404	Devuelve error sin cambios
CorreoDuplicado	Correo ya registrado	409	Rechaza el registro
DatosInvalidos	Campos faltantes o con formato incorrecto	422	Devuelve detalle de los campos con error
ClienteSuspendido	Operación sobre cliente suspendido	403	Rechaza la operación
ServicioNoDisponible	Falla de base de datos o Autenticación	503	Reintento; sin alta parcial (rollback)

5. Restricciones

- Registro en menos de 1 s (p95).
- Datos personales cifrados en reposo.
- Eliminación lógica (no se borran clientes con órdenes).

6. Vistas de IEEE 1016 usadas

- Composición: diagrama de paquetes.
- Interfaz: contrato OpenAPI.
- Interacción: secuencia de registro de cliente.
- Información: modelo de datos Cliente–Dirección.
- Dinámica de estados: ciclo de vida del cliente.

2.5.6. Ficha de diseño del módulo Gestión de proveedores

MOD-PRV v1.0 · Área 3 · IEEE 1016-2009

Código	MOD-PRV v1.0
Responsabilidad	Administrar el registro de proveedores, sus productos suministrados y las órdenes de reabastecimiento para mantener el inventario.
Estados	Proveedor: En evaluación → Activo → Suspendido. Reabastecimiento: Solicitada → Confirmada → Recibida (o Rechazada)

1. Atributos

Entidad	Atributo	Tipo	Restricción
Proveedor	idProveedor	UUID, PK	Único
Proveedor	razonSocial / nit	varchar(120) / varchar(20)	NIT único
Proveedor	contacto / correo / telefono	varchar	Correo válido
Proveedor	estado	enum	Activo, En evaluación, Suspendido
Proveedor	calificacion	decimal(2,1)	Entre 0.0 y 5.0
ProductoProveedor	idProveedor / idProducto	FK / FK	Par único
ProductoProveedor	costoUnitario / tiempoEntregaDias	decimal / int	Costo mayor que 0
Reabastecimiento	idSolicitud / cantidad / estado / fecha	PK / int / enum / timestamp	Solicitada, Confirmada, Recibida, Rechazada

2. Interfaces

Provistas	POST /v1/proveedores GET /v1/proveedores/{id} PUT /v1/proveedores/{id} GET /v1/proveedores/{id}/productos POST /v1/proveedores/{id}/reabastecimientos Evento: ReabastecimientoSolicitado
Requeridas	Autenticación (validarToken, credenciales de proveedor) Catálogo e inventario (registrarEntrada, consultar stock mínimo)

3. Dependencias

Detalle	Internas: Autenticación, Catálogo e inventario. Externas: MySQL (proveedores y reabastecimientos)

4. Manejo de excepciones

Excepción	Causa	HTTP	Acción del módulo
ProveedorNoEncontrado	El id no existe	404	Devuelve error sin cambios
NitDuplicado	El NIT ya está registrado	409	Rechaza el alta
ProveedorSuspendido	Solicitud a un proveedor suspendido	403	Rechaza el reabastecimiento
ProductoNoSuministrado	El proveedor no vende ese producto	422	Rechaza la solicitud
ServicioNoDisponible	Falla de Catálogo o notificaciones	503	Reintento; la solicitud queda pendiente

5. Restricciones

- Consulta de proveedor en menos de 500 ms (p95).
- Un reabastecimiento recibido actualiza el inventario de forma transaccional.
- Historial de proveedores conservado para auditoría.

6. Vistas de IEEE 1016 usadas

- Composición: diagrama de paquetes.
- Interfaz: contrato OpenAPI.
- Interacción: secuencia de reabastecimiento.
- Información: modelo de datos Proveedor–Producto.
- Dinámica de estados: ciclo del reabastecimiento.

2.5.7. Ficha de diseño del módulo Pagos

MOD-PAG v1.0 · IEEE 1016-2009

Código	MOD-PAG v1.0
Responsabilidad	Procesar y registrar los pagos de una cotización u orden mediante el patrón Strategy (ProcesadorPago), sin crear el documento de envío cuando el pago es rechazado.
Estados	Pendiente → Aprobado | Rechazado ; Aprobado → Reembolsado

1. Atributos

Entidad	Atributo	Tipo	Restricción
Pago	idPago	UUID, PK	Único
Pago	id_cotizacion	FK	Debe existir y estar vigente
Pago	metodo	enum	Efectivo, ReferenciaBancaria, Tarjeta
Pago	monto / moneda	decimal(10,2) / char(3)	Monto igual al total de la cotización
Pago	estado	enum	Pendiente, Aprobado, Rechazado, Reembolsado
Pago	referencia / fecha	varchar(60) / timestamp	Referencia de la pasarela o del banco
ResultadoPago	aprobado / referencia / mensaje	bool / varchar / varchar	Objeto devuelto por cada estrategia
DatosPago	numeroTarjetaToken / vencimiento	varchar / varchar	Nunca se guarda el número completo

2. Interfaces

Provistas	POST /v1/pagos/procesar GET /v1/pagos/{id} GET /v1/cotizaciones/{id}/pagos POST /v1/pagos/{id}/reembolso Eventos: PagoAprobado, PagoRechazado
Requeridas	Autenticación y seguridad (validarToken, permisos) Gestión de órdenes / cotizaciones (existencia, estado y total) PasarelaPago (simulación, solo para tarjeta)

3. Dependencias

Detalle	Internas: Autenticación, Cotizaciones y órdenes. Externas: PasarelaPago (simulada), PostgreSQL, broker de eventos. Componentes: ProcesadorPago.php (reutilizado), EfectivoStrategy, ReferenciaBancariaStrategy, TarjetaStrategy, PasarelaPago.

4. Manejo de excepciones

Excepción	Causa	HTTP	Acción del módulo
CotizacionNoEncontrada	La cotización no existe	404	Rechaza el pago
SinPermiso	El usuario no tiene permisos sobre la cotización	403	Rechaza y registra el intento
EstadoNoPermitido	Cotización rechazada o convertida a venta	422	Rechaza el pago
MetodoNoSoportado	Método de pago desconocido	422	Rechaza la solicitud
PagoRechazado	Tarjeta rechazada por la pasarela	400	Registra el pago como Rechazado; no se crea documento de envío
MontoInconsistente	El monto no coincide con el total	422	Rechaza el pago

5. Restricciones

- Autorización de pago en menos de 3 s (p95).
- Datos de tarjeta tokenizados; nunca se almacenan completos.
- La pasarela es solo simulación; no se realizan cargos reales.

6. Vistas de IEEE 1016 usadas

- Composición: diagrama de paquetes.
- Interfaz: contrato OpenAPI.
- Interacción: secuencia de procesamiento de pago.
- Información: modelo de datos Pago.
- Dinámica de estados: ciclo de vida del pago.

2.5.8. Ficha de diseño del módulo Envíos

MOD-ENV v1.0 · IEEE 1016-2009

Código	MOD-ENV v1.0
Responsabilidad	Crear el documento de envío de una cotización con pago aprobado (uno solo por cotización) y dar seguimiento a la entrega hasta el cliente.
Estados	Pendiente → Asignado → En camino → Entregado (o Fallido)

1. Atributos

Entidad	Atributo	Tipo	Restricción
DocumentoEnvio	id_documento	UUID, PK	Único
DocumentoEnvio	id_cotizacion	FK, UNIQUE	Un solo documento por cotización
DocumentoEnvio	idPago	FK	Debe estar Aprobado
DocumentoEnvio	direccionEntrega	texto	Obligatoria
DocumentoEnvio	transportista / numeroGuia	varchar	Guía única
DocumentoEnvio	estado	enum	Pendiente, Asignado, En camino, Entregado, Fallido
DocumentoEnvio	fechaCreacion / fechaEstimada	timestamp / date	Fecha estimada posterior a la creación
SeguimientoEnvio	idDocumento / estado / fecha / nota	FK / enum / timestamp / texto	Solo inserción

2. Interfaces

Provistas	POST /v1/envios GET /v1/envios/{id} GET /v1/cotizaciones/{id}/envio GET /v1/envios/{id}/seguimiento PATCH /v1/envios/{id}/estado Eventos: EnvioCreado, EnvioEntregado
Requeridas	Autenticación y seguridad (validarToken, permisos) Pagos (verificar que el pago esté Aprobado) Gestión de órdenes / cotizaciones (existencia, estado y dirección) Notificaciones (aviso de envío)

3. Dependencias

Detalle	Internas: Autenticación, Pagos, Cotizaciones y órdenes, Notificaciones. Externas: API del transportista, PostgreSQL, broker de eventos.

4. Manejo de excepciones

Excepción	Causa	HTTP	Acción del módulo
CotizacionNoEncontrada	La cotización no existe	404	Rechaza la solicitud
SinPermiso	El usuario no tiene permisos	403	Rechaza y registra el intento
EstadoNoPermitido	Cotización rechazada o convertida a venta	422	Rechaza la solicitud
DocumentoDuplicado	La tabla ya contiene el id_cotizacion	409	Rechaza y devuelve el documento existente
PagoNoAprobado	El pago fue rechazado o no existe	400	No crea documento de envío
TransportistaNoDisponible	Timeout de la API del transportista	503	Reintento con backoff; el envío queda Pendiente

5. Restricciones

- Un solo documento de envío por cotización (restricción UNIQUE en id_cotizacion).
- Creación del documento y registro de seguimiento en una sola transacción.
- Máximo 2 reintentos de entrega fallida.

6. Vistas de IEEE 1016 usadas

- Composición: diagrama de paquetes.
- Interfaz: contrato OpenAPI.
- Interacción: secuencia de creación del documento de envío.
- Información: modelo de datos DocumentoEnvio.
- Dinámica de estados: ciclo de vida del envío.

2.4 Gestión Ágil de Proyectos

2.4.1 Marco de trabajo (Scrum/Kanban) y rol del Product Owner.

2.4.1.1. Propósito del rol

El Analista / Product Owner (PO) es el responsable de que el sistema construya lo que el negocio realmente necesita. Representa a los interesados (clientes, vendedores, administradores y proveedores), traduce sus necesidades en requerimientos verificables y decide el orden en que se desarrollan, para maximizar el valor entregado en cada iteración. En este proyecto el rol combina dos perspectivas:

- Analista de requisitos: elicita, redacta, valida y mantiene la trazabilidad de los requerimientos.
- Product Owner: es dueño del Product Backlog, lo prioriza y acepta o rechaza los entregables.

2.4.1.1. Responsabilidades

Elicitación y análisis

- Identificar a los interesados y recoger sus necesidades mediante entrevistas, observación y revisión de procesos de venta, pago y envío.
- Definir el alcance del sistema y mantener claros los límites de cada módulo (Autenticación, Catálogo, Inventario, Cotizaciones, Clientes, Proveedores, Pagos, Envíos).
- Documentar las reglas de negocio, por ejemplo: un solo documento de envío por cotización o no procesar pago-envío si la cotización está rechazada.

Especificación

- Redactar y mantener el SRS conforme a ISO/IEC/IEEE 29148, con requerimientos funcionales (REQ-FN) y no funcionales (REQ-NF) en formato verificable: condición, sujeto y acción, con un valor medible.
- Escribir las historias de usuario (US-xx) con sus criterios de aceptación y mantener la matriz de trazabilidad historia → requerimiento → prioridad.
- Garantizar que cada requerimiento sea necesario, no ambiguo, verificable y consistente con los demás.

Gestión del Product Backlog

- Crear, refinar y ordenar el backlog, con épicas e historias en el tablero del proyecto.
- Priorizar con MoSCoW (Debe, Debería, Podría) según valor de negocio, riesgo y dependencias entre módulos.
- Definir, junto con el equipo, qué significa que una historia esté terminada (definición de hecho).

Validación y aceptación

- Participar en las revisiones de cada iteración y aceptar o rechazar los incrementos contra los criterios de aceptación.
- Gestionar los cambios de requerimientos: evaluar su impacto, registrarlos y actualizar el SRS y el backlog.

Enlace con el diseño

- Aclarar dudas del equipo de desarrollo y revisar que las fichas de diseño (IEEE 1016) y los diagramas UML cubran los requerimientos acordados.

2.4.2 Product Backlog con Historias de Usuario (INVEST / 3C)

2.4.2.1. Vínculo al tablero ágil

https://github.com/users/Dailin-Xepuxtian/projects/2

2.4.2.2. Cómo leer este backlog

- **Card (tarjeta):** la historia en una frase: *Como [rol], quiero [necesidad], para [beneficio]*.
- **Conversation (conversación):** los puntos que el PO y el equipo deben aclarar antes de construir: reglas, casos límite y decisiones.
- **Confirmation (confirmación):** criterios de aceptación verificables con el formato *Dado que… cuando… entonces….*
- **INVEST:** Independiente, Negociable, Valiosa, Estimable, Pequeña (Small) y Testeable.
- **MoSCoW:** Must (Debe), Should (Debería), Could (Podría). **SP:** puntos de historia.

2.4.2.3. Resumen del backlog

ID	Historia	Épica	MoSCoW	SP	Estado en el tablero	Issue
US-01	Consultar catálogo de productos	EP-01 Catálogo y disponibilidad	Must	3	Done	#1
US-02	Buscar productos	EP-01 Catálogo y disponibilidad	Must	3	Done	#2
US-03	Filtrar productos	EP-01 Catálogo y disponibilidad	Should	5	Done	#4
US-04	Consultar detalle del producto	EP-01 Catálogo y disponibilidad	Must	3	Ready	#5
US-05	Agregar productos al carrito	EP-01 Catálogo y disponibilidad	Must	5	Ready	#6
US-06	Modificar cantidad de productos	EP-02 Carrito de compras	Must	3	Ready	#7
US-07	Eliminar productos del carrito	EP-02 Carrito de compras	Must	2	Backlog	#8
US-08	Consultar resumen de compra	EP-03 Gestión del proceso de venta	Must	5	Done	#9
US-09	Registrar datos de entrega	EP-03 Gestión del proceso de venta	Must	5	Backlog	#10
US-10	Seleccionar método de pago	EP-04 Pagos	Must	por confirmar	Backlog	por verificar
US-11	Confirmar compra	EP-04 Pagos	Must	3	Backlog	#12
US-12	Generar número de pedido	EP-05 Gestión del pedido	Must	3	Ready	#13

US-13	Consultar estado del pedido	EP-05 Gestión del pedido	Should	5	Backlog	#14
US-14	Consultar historial de compras	EP-06 Historial y seguimiento	Could	5	Backlog	#15
US-15	Consultar ventas realizadas	EP-03 Gestión del proceso de venta	Must	5	Backlog	#16

2.4.2.4 Validación INVEST

La tabla marca ✓ cuando la historia cumple el criterio y ⚠ cuando cumple con una observación que conviene atender antes de construirla.

Historia	I	N	V	E	S	T	Observación
US-01	✓	✓	✓	✓	✓	✓	Sin observaciones.
US-02	✓	✓	✓	✓	✓	✓	Sin observaciones.
US-03	✓	✓	✓	✓	✓	✓	Sin observaciones.
US-04	✓	✓	✓	✓	✓	✓	Sin observaciones.
US-05	⚠	✓	✓	✓	✓	✓	Depende de la consulta de existencias (Inventario).
US-06	⚠	✓	✓	✓	✓	✓	Depende de US-05 (el producto debe estar en el carrito).
US-07	⚠	✓	✓	✓	✓	✓	Depende de US-05.
US-08	⚠	✓	✓	✓	✓	✓	Depende del carrito (US-05 a US-07).
US-09	✓	✓	✓	✓	⚠	✓	Con 5 SP está en el límite; si crece, separar la validación de campos.
US-10	⚠	✓	✓	⚠	✓	✓	Depende de US-09 y aún no tiene SP visibles en el tablero.
US-11	⚠	✓	✓	✓	✓	✓	Depende de US-10 y de la pasarela simulada.
US-12	⚠	✓	✓	✓	✓	✓	Depende de que la compra esté confirmada (US-11).
US-13	⚠	✓	✓	✓	✓	✓	Depende de US-12.
US-14	⚠	✓	✓	✓	⚠	✓	Requiere inicio de sesión (US-18*) y 5 SP.
US-15	⚠	✓	✓	✓	⚠	✓	Requiere sesión de administrador (US-18*) y 5 SP.

2.4.2.5. Definition of Ready (DoR)

Una historia solo pasa a la columna **Ready** cuando cumple todo lo siguiente:

- ☐ Está redactada como Card: *Como [rol], quiero [necesidad], para [beneficio]*.
- ☐ Tiene al menos dos criterios de aceptación en formato *Dado / Cuando / Entonces*, medibles y verificables.
- ☐ Los puntos de la Conversation están resueltos o registrados como decisiones del PO.
- ☐ Cumple INVEST; las dependencias están identificadas y resueltas o planificadas.
- ☐ Está estimada en SP y no supera 5 SP (si los supera, se divide).
- ☐ Tiene prioridad MoSCoW y está asociada a una épica.
- ☐ Está trazada a su requerimiento (REQ-FN-###) del SRS.
- ☐ Se conocen las reglas de negocio y los módulos afectados (ficha de diseño correspondiente).

- ☐ El equipo la entiende y está de acuerdo en poder terminarla en una iteración.

2.4.2.6. Definition of Done (DoD)

Una historia solo pasa a la columna **Done** cuando cumple todo lo siguiente:

- ☐ Todos los criterios de aceptación (Confirmation) se probaron y pasan.
- ☐ El código fue revisado por otra persona (pull request aprobado) e integrado a la rama principal sin romper lo existente.
- ☐ Se ejecutaron las pruebas funcionales y las de los requerimientos no funcionales que le aplican (tiempos de respuesta, HTTPS, contraseñas cifradas, diseño adaptable).
- ☐ No quedan defectos críticos abiertos sobre la historia.
- ☐ La documentación está actualizada: SRS y matriz de trazabilidad, ficha de diseño del módulo y diagramas UML si cambiaron.
- ☐ El PO revisó la funcionalidad en una demostración y la aceptó.
- ☐ El issue está enlazado a su pull request, cerrado y movido a Done en el tablero.

Vínculos:

Mi-proyecto-sistema2/docs/uml

Mi-proyecto-sistema2/docs/requirements

3. EVALUACIÓN ARQUITECTÓNICA, C4 Y PERSISTENCIA

3.1 Evaluación de Estilos Arquitectónicos y Escenarios de Calidad (ISO/IEC 25010)

3.1.1 Atributos de calidad priorizados (ISO/IEC 25010)

Característica ISO 25010	Subcaracterística relevante	Por qué importa en Giganet
Eficiencia de desempeño	Comportamiento temporal	El catálogo y las cotizaciones se consultan constantemente
Mantenibilidad	Modularidad, modificabilidad, capacidad de ser probado	El equipo es pequeño y el alcance crecerá (pagos, envíos, inventario)
Fiabilidad	Madurez, tolerancia a fallos	No debe haber sobreventa ni reservas huérfanas
Seguridad	Autenticidad, confidencialidad	Hay usuarios con roles y datos de clientes (NIT, correo)
Adecuación funcional	Corrección funcional	IVA 12 %, reglas de negocio RN-01 a RN-05

Escalabilidad se evalúa dentro de *Eficiencia de desempeño* (subcaracterística *capacidad*) y se trata aquí como atributo propio porque el documento lo exige.

3.1.2. Escenarios de calidad

Los objetivos medibles son propuestas de diseño para validar la arquitectura; se ajustan con pruebas reales.

ID	Atributo	Fuente	Estímulo	Entorno	Respuesta	Medida
ES-01	Mantenibilidad	Desarrollador	Agregar un nuevo método de pago	Desarrollo	Se añade una clase Strategy y se registra, sin tocar controladores	≤ 1 archivo existente modificado
ES-02	Desempeño	Vendedor	Consulta GET /productos con filtros	Operación normal, 50 usuarios concurrentes	Respuesta desde caché Cache-Aside (hoy FileCache; Redis como objetivo, ADR-004)	p95 < 500 ms

ES-03	Escalabilidad	Aumento de demanda	5 veces el tráfico en cotizaciones	Temporada alta	Se replican instancias de la API (sin estado, JWT)	Sin cambios en la lógica de dominio
ES-04	Fiabilidad	Dos vendedores	Reservan la última unidad a la vez	Operación normal	Solo una reserva queda activa	0 sobreventas
ES-05	Seguridad	Cliente no autenticado	Llama a POST /cotizaciones sin token	Operación normal	La API rechaza la solicitud	100 % de respuestas 401

3.1.3. Comparativa de estilos

Escala: **Alta / Media / Baja** (qué tan bien cumple el estilo cada atributo).

Atributo	**Capas (monolito modular)**	**Repositorio (datos centralizados)**	**Microservicios**	**Orientada a eventos (EDA)**
Escalabilidad	Media (se replica el conjunto)	Baja (cuello de botella en el repositorio)	Alta (por servicio)	Alta (productores y consumidores desacoplados)
Mantenibilidad	Alta con equipo pequeño	Media (acoplamiento por el esquema)	Media con equipo pequeño, alta en equipos grandes	Media (flujo difícil de seguir)
Desempeño	Alta (llamadas en proceso)	Alta	Media (latencia de red)	Alta en rendimiento, latencia variable
Fiabilidad / consistencia	Alta (ACID en MySQL)	Alta	Eventual, requiere sagas	Eventual, requiere idempotencia
Complejidad operativa	Baja	Baja	Alta (orquestación, observabilidad)	Alta (broker, reintentos)
Seguridad	Un único punto de control	Un único punto de control	Superficie mayor, más puntos de autenticación	Superficie mayor, mensajes por proteger
Costo y tiempo de entrega	Bajo	Bajo	Alto	Alto

3.1.4. Resultado de la evaluación

Se elige **monolito modular en capas** (Controller → Service → Repository) como estilo base. Cumple mejor ES-01 y ES-05, mantiene la consistencia ACID que exige ES-04 y tiene el menor costo operativo para el equipo.

Los estilos distribuidos quedan como **ruta de evolución**:

- Eventos internos con el patrón Observer (cambios de estado de cotización y reservas).
- Extraer inventario y notificaciones como servicios si la carga lo justifica (ES-03).
- EDA con un broker solo para los flujos asíncronos (envíos, notificaciones).
- El estilo Repositorio como arquitectura se descarta; el patrón Repository sí se usa dentro de las capas (ADR-007).

3.2 Registros de Decisiones de Arquitectura (ADR - ISO/IEC/IEEE 42010:2011)

Formato basado en **ISO/IEC/IEEE 42010:2011**: cada ADR identifica interesados, preocupaciones, contexto, decisión, alternativas y consecuencias.

Índice

ID	Decisión	Estado
ADR-001	Monolito modular en capas	Aceptado
ADR-002	MySQL como base transaccional	Aceptado
ADR-003	Persistencia políglota como arquitectura objetivo	Propuesto
ADR-004	Caché Cache-Aside con Redis y TTL	Aceptado
ADR-005	Autenticación JWT sin estado	Aceptado
ADR-006	Consistencia eventual con saga coreografiada	Propuesto
ADR-007	Patrones GoF del proyecto	Aceptado

Estados posibles

Propuesto → Aceptado → Reemplazado / Obsoleto. «Aceptado» significa decidido e implementado o en implementación; «Propuesto» es arquitectura objetivo documentada pero no implementada.

ADR-001: Monolito modular en capas

Estado: Aceptado

Fecha: 2026-10-21

Preocupaciones (ISO 42010): mantenibilidad, desempeño, costo de entrega

Contexto

El módulo de ventas y cotizaciones lo desarrolla un equipo pequeño en PHP 8 sin framework. El punto de entrada es app.php, con una tabla de rutas que despacha a controladores. El resto

del código ya está separado en controllers/, services/ y models/repositories/, con DTOs de entrada y salida.

Decisión

Se adopta un **monolito modular en capas**: Controller (HTTP) → Service (reglas de negocio) → Repository (acceso a datos). Los controladores no acceden a la base de datos y los repositorios no contienen reglas de negocio.

Alternativas consideradas

Repositorio / datos centralizados: descartado por acoplamiento al esquema y cuello de botella.

Microservicios: descartado por costo operativo y complejidad desproporcionados al tamaño del equipo y del sistema.

EDA: descartado como estilo base; se reserva para flujos asíncronos (ver ADR-006).

Consecuencias

Positivas: despliegue simple, transacciones ACID, pruebas de integración por capa, menor latencia.

Negativas: se escala el conjunto completo; un módulo defectuoso puede afectar a todos. Además, los servicios crean sus dependencias directamente con new en lugar de recibirlas por inyección de dependencias, lo que dificulta las pruebas unitarias aisladas (no se pueden sustituir los repositorios por dobles de prueba).

Mitigación: límites claros entre módulos (productos, cotizaciones, ventas, inventario) que permitan extraerlos más adelante. Como mejora futura, inyectar los repositorios por constructor en los servicios, de modo que sea posible probarlos de forma unitaria sin MySQL.

ADR-002: MySQL como base de datos transaccional principal

Estado: Aceptado

Fecha: 2026-10-21

Preocupaciones (ISO 42010): integridad de datos, fiabilidad, curva de aprendizaje

Contexto

giganet_db modela clientes, productos, inventario, cotizaciones, ventas y pagos con claves foráneas entre tablas (por ejemplo cotizaciones → detalle_cotizaciones, ventas → pagos). Las operaciones críticas (crear una cotización con sus detalles) requieren transacciones.

Decisión

Se usa **MySQL** como almacén relacional principal. El documento de requerimientos menciona PostgreSQL como referencia; aquí se documenta la desviación y su justificación.

Alternativas consideradas

PostgreSQL: mejor soporte de tipos avanzados (JSONB, arrays) y extensiones, pero el entorno de desarrollo del equipo (phpMyAdmin) y su experiencia ya están en MySQL, y el modelo actual no necesita esas capacidades.

Consecuencias

Positivas: integridad referencial, transacciones, herramientas conocidas por el equipo, sin costo de migración.

Negativas: menos flexibilidad para datos semiestructurados; esos casos se atienden con MongoDB (ADR-003).

Nota: el cambio a PostgreSQL es viable porque el acceso a datos está aislado en repositorios.

ADR-003: Persistencia políglota como arquitectura objetivo

Estado: Propuesto (MongoDB previsto parcialmente en el esquema)

Fecha: 2026-10-21
Preocupaciones (ISO 42010): escalabilidad, desempeño, adecuación del modelo de datos
Contexto
No todos los datos tienen la misma forma ni el mismo patrón de acceso. La tabla ventas ya incluye id_envio_mongodb, una referencia a un documento externo. Las reservas de inventario tienen fecha_expiracion y estados activa/completada/cancelada/expirada, que se parecen a datos con caducidad. La relación producto–proveedor es de muchos a muchos.
Decisión
Se define el siguiente reparto de responsabilidades:

Almacén	Tipo	Uso en Giganet
MySQL	Relacional	Cotizaciones, ventas, pagos, clientes, productos, usuarios
MongoDB	Documental	Datos de envío (documentos de estructura variable), referenciados desde ventas.id_envio_mongodb
Redis	En memoria	Caché del catálogo (ADR-004) y reservas temporales con TTL
Neo4j	Grafos	Relaciones producto–proveedor y recomendaciones por historial de compra

MySQL sigue siendo la **fuente de verdad** de los datos transaccionales; los demás almacenes complementan.
Alternativas consideradas
Solo MySQL: más simple, pero sin ventajas para documentos flexibles, caducidad automática ni consultas de relaciones profundas.
Consecuencias
Positivas: cada dato en el almacén que mejor lo atiende; mejor desempeño de lectura.
Negativas: más componentes que operar, consistencia eventual entre almacenes (ADR-006), mayor costo de pruebas.
Alcance actual: MySQL es la fuente de verdad. La caché Cache-Aside ya está implementada con archivos (FileCache, ver ADR-004); Redis sigue como siguiente paso de mayor valor. MongoDB y Neo4j son arquitectura objetivo.

ADR-004: Caché Cache-Aside con Redis y expiración (TTL)
Estado: Aceptado (Cache-Aside implementado con caché en archivos en GET /productos y GET /categorias; Redis sigue como arquitectura objetivo)
Fecha: 2026-10-21
Preocupaciones (ISO 42010): desempeño, escalabilidad (escenarios ES-02 y ES-03)
Contexto
GET /productos y GET /categorias son las lecturas más frecuentes y cambian poco comparado con su consumo.
Decisión
Se aplica el patrón **Cache-Aside** en la capa de servicios:
El servicio consulta Redis con una clave.
Si hay acierto (*hit*), responde desde Redis.
Si hay fallo (*miss*), consulta el repositorio MySQL, guarda el resultado en Redis con TTL y responde.

Al modificar un producto o su inventario, se invalida la clave afectada.

```
public function listar(array $filtros): array
{
  clave='productos:'.md5(jsonencode(filtros));
  $cache = this->redis->get(clave);
  if ($cache !== false) {
     return json_decode($cache, true);
  }
  $datos = this->productoRepository->listar(filtros);
  this->redis->setex(clave, 300, json_encode($datos));
  return $datos;
}
```

Clave	TTL propuesto	Justificación
productos:{hash filtros}	300 s	El inventario cambia con las ventas
categorias	3600 s	Cambia muy rara vez
reserva:{id}	hasta fecha_expiracion	Coincide con la expiración de la reserva

Alternativas consideradas

Write-Through: mantiene la caché siempre actual, pero escribe en dos sitios en cada cambio; innecesario para este volumen.

Sin caché: viable hoy, pero no cumple ES-02 con crecimiento.

Consecuencias

Positivas: menor carga en MySQL y menor latencia en lecturas.

Negativas: posibles datos desactualizados hasta el vencimiento del TTL; hay que invalidar correctamente. Si Redis falla, el servicio debe seguir funcionando consultando MySQL (degradación controlada).

Implementación actual: ProductoService usa CacheAside con FileCache (src/patterns/cache/), que guarda en storage/cache/ con TTL de 300 s (productos) y 3600 s (categorías). No requiere Redis. Pasar a Redis consiste en sustituir FileCache por RedisCache, ya que ambos implementan CacheInterface. Crear una cotización no invalida la caché, por lo que el stock mostrado puede estar atrasado hasta 5 minutos.

ADR-005: Autenticación con JWT sin estado

Estado: Aceptado

Fecha: 2026-10-21

Preocupaciones (ISO 42010): seguridad, escalabilidad (ES-03, ES-05)

Contexto

La API se consume desde un prototipo web en otro origen. La tabla de rutas de app.php marca por ruta si se requiere token: POST auth/login es pública y el resto exige JWT. Existe un servicio Jwt y un AuthService.

Decisión

Se usa **JWT firmado** emitido en POST /auth/login y enviado en la cabecera Authorization. El servidor no guarda sesiones.

Alternativas consideradas

Sesiones PHP en servidor: simples, pero atan al cliente a una instancia y dificultan replicar la API.
OAuth 2.0 con proveedor externo: excesivo para un sistema interno.
Consecuencias
Positivas: la API es replicable sin compartir sesiones; validación rápida en cada solicitud.
Negativas: un token emitido no se puede revocar antes de que expire.
Mitigaciones: expiración corta, secreto en variables de entorno (Env.php), y restringir CORS al origen del prototipo en producción (hoy permite cualquier origen).

ADR-006: Consistencia eventual con saga coreografiada
Estado: Propuesto
Fecha: 2026-10-21
Preocupaciones (ISO 42010): fiabilidad (ES-04), consistencia entre almacenes
Contexto
El flujo cotización → venta → reserva de inventario → pago toca varias tablas y, en la arquitectura objetivo, varios almacenes (MySQL, MongoDB, Redis). No existe una transacción ACID que abarque todos.
Decisión
Dentro de MySQL: una sola transacción ACID para operaciones atómicas (cotización + detalle_cotizaciones; venta + detalle_ventas).
Entre almacenes: consistencia eventual mediante una **saga coreografiada**; cada paso publica un evento y define su compensación.

Paso	Evento	Estado resultante	Compensación
1. Se crea la cotización	CotizacionCreada	borrador / enviada	Anular cotización
2. Se aprueba	CotizacionAprobada	aprobada, reserva activa	Reserva cancelada
3. Se convierte en venta	VentaCreada	convertida_a_venta, venta pendiente	Venta anulada, reserva cancelada
4. Se registra el pago	PagoVerificado	venta completada, reserva completada	—
4b. Pago rechazado o reserva vencida	PagoRechazado / ReservaExpirada	pago rechazado, reserva expirada	Venta anulada, stock liberado

Los consumidores deben ser **idempotentes** (procesar dos veces el mismo evento no duplica efectos).
Alternativas consideradas
Transacción distribuida (2PC): no está soportada de forma práctica entre MySQL, MongoDB y Redis y reduce la disponibilidad.
Saga orquestada: centraliza el control, pero agrega un componente más; innecesario con cuatro pasos.
Consecuencias

Positivas: sin bloqueos largos; los fallos se compensan de forma explícita. **Negativas:** hay ventanas breves de inconsistencia; hay que diseñar eventos, reintentos e idempotencia.

ADR-007: Patrones GoF del proyecto
Estado: Aceptado (Repository integrado; Strategy, Observer, Gateway y Circuit Breaker implementados como módulo independiente)
Fecha: 2026-10-21
Preocupaciones (ISO 42010): mantenibilidad (ES-01), fiabilidad
Contexto
El enunciado exige aplicar de forma explícita los patrones Strategy, Repository, Gateway, Observer y Circuit Breaker.
Decisión

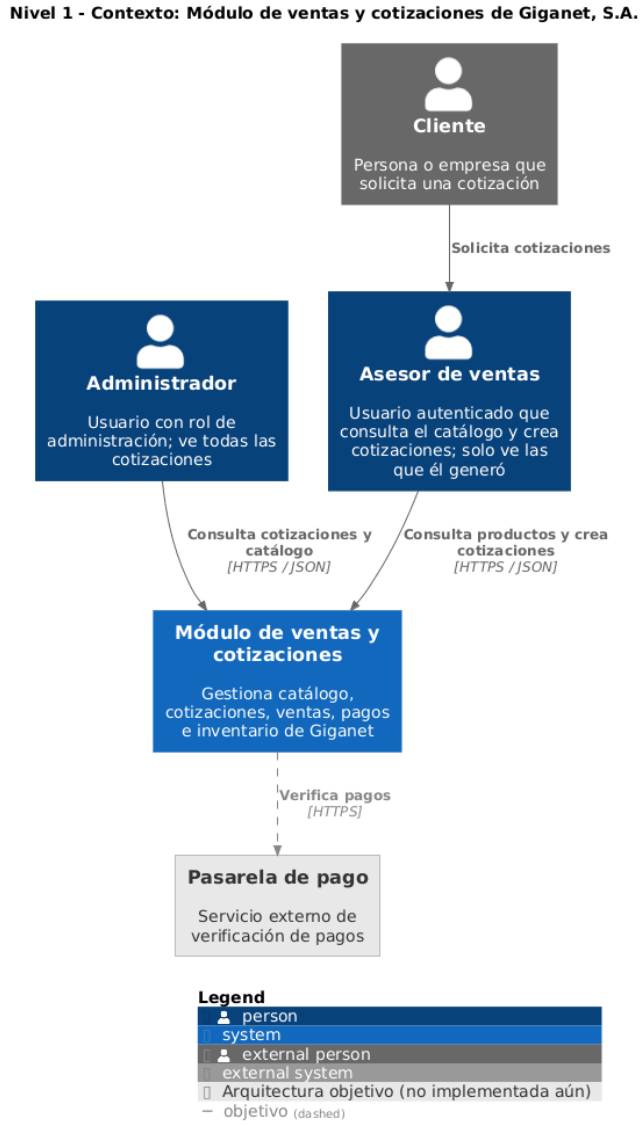
Patrón	Dónde se aplica	Estado
Repository	ClienteRepository, CotizacionRepository, ProductoRepository, UsuarioRepository aíslan el acceso a MySQL	Implementado e integrado en los endpoints
Strategy	Verificación de pago según el método (efectivo, transferencia, tarjeta_credito) en src/patterns/strategy/	Implementado (módulo independiente en src/patterns/, aún sin conectar a endpoints)
Observer	Cambio de estado de la cotización notifica a inventario (reserva) y a la bitácora, en src/patterns/observer/	Implementado (módulo independiente en src/patterns/, aún sin conectar a endpoints)
Gateway	PasarelaPagoSimulada y PasarelaPagoConCircuitBreaker encapsulan el servicio externo de pago, en src/patterns/gateway/	Implementado (módulo independiente en src/patterns/, aún sin conectar a endpoints)
Circuit Breaker	CircuitBreaker en src/patterns/resilience/: tras N fallos seguidos abre el circuito y responde con error controlado	Implementado (módulo independiente en src/patterns/, aún sin conectar a endpoints)

Verificación: los cuatro módulos independientes se probaron con tools/demo_patrones.php, que no requiere MySQL ni Redis.
Alternativas consideradas
Lógica condicional en los servicios (if/else por método de pago): más rápido al inicio, pero incumple ES-01.
Consecuencias

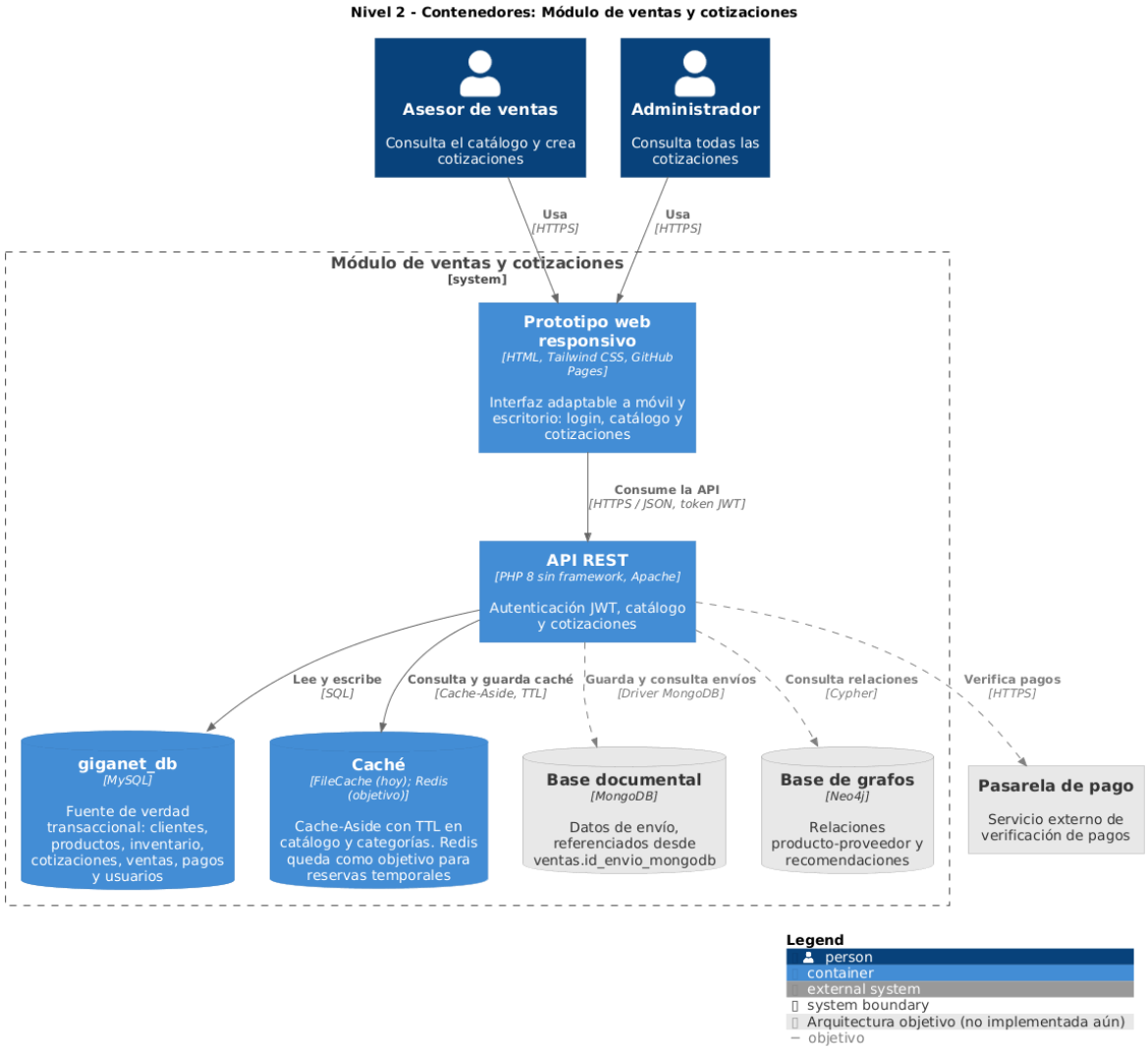
Positivas: nuevas variantes (otro método de pago, otro canal de envío) se agregan sin modificar código existente.
Negativas: más clases e interfaces; se justifica por el crecimiento previsto. Mientras los módulos no estén conectados a los endpoints, su efecto en el sistema real es nulo y solo se demuestran con el script.

3.3 Documentación de la Arquitectura mediante el Modelo C4 (4 Niveles)

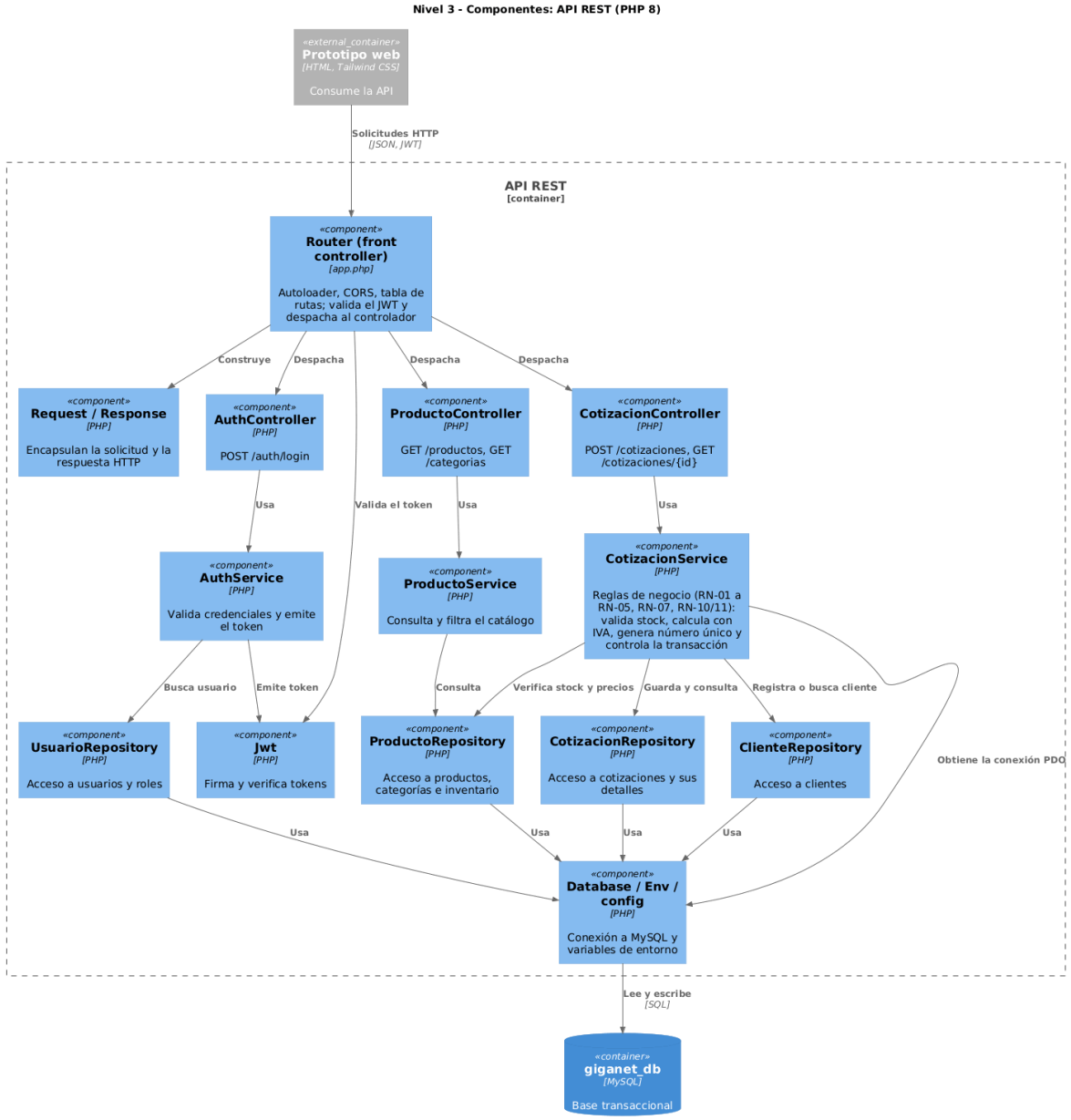
Nivel 1 – Contexto



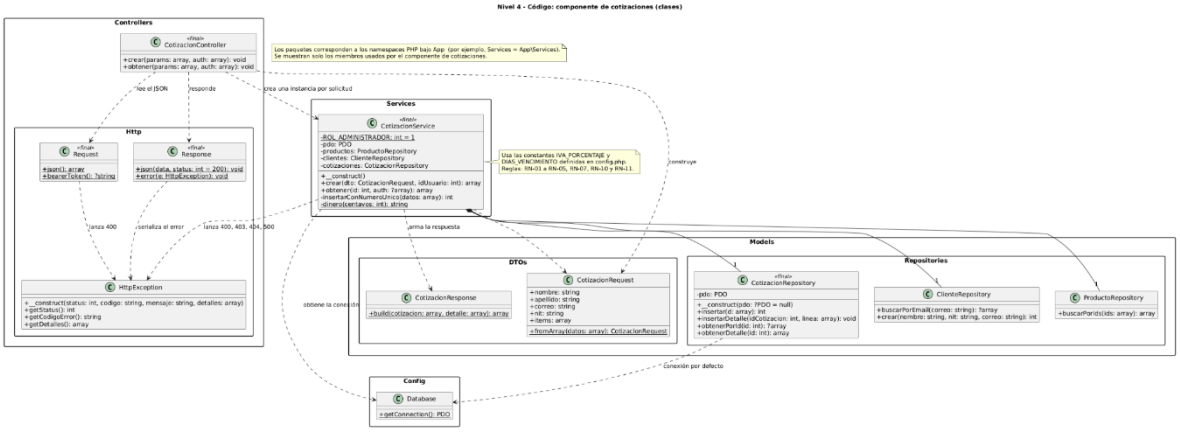
Nivel 2 – Contenedores



Nivel 3 – Componentes



Nivel 4 – Código



3.4 Arquitectura de Persistencia Políglota (PostgreSQL, MongoDB, Redis, Neo4j) y Cache-Aside

Sistema: Módulo de ventas y cotizaciones de Giganet, S.A.
Relacionado con: ADR-003, ADR-004, ADR-006 y ADR-007 (docs/architecture/adrs/).

1. Persistencia políglota

MySQL es la fuente de verdad. Los demás almacenes son la arquitectura objetivo (ADR-003).

Almacén	Tipo	Uso en Giganet	Estado
MySQL (giganet_db)	Relacional	Cotizaciones, ventas, pagos, clientes, productos, usuarios	Implementado
MongoDB	Documental	Datos de envío, referenciados desde ventas.id_envio_mongodb	Objetivo
Redis	En memoria	Caché del catálogo y estado del Circuit Breaker	Adaptador escrito (RedisCache); sin conectar. En uso: FileCache
Neo4j	Grafos	Relaciones producto–proveedor y recomendaciones	Objetivo

2. Caché Cache-Aside con expiración (TTL)

Flujo: consultar caché → si hay dato, responderlo → si no, consultar MySQL, guardar con TTL y responder. Al modificar un dato en MySQL se invalida su clave. Si la caché falla, el sistema sigue funcionando con MySQL.

Clave	TTL	Justificación
productos:{hash de filtros}	300 s	El inventario cambia con las ventas
categorias	3600 s	Cambia muy rara vez

Código: src/patterns/cache/ (CacheInterface, ArrayCache, RedisCache, CacheAside). FileCache (caché en archivos, storage/cache/) está conectado a ProductoService para el catálogo y las categorías, con TTL de 300 s y 3600 s; Redis queda como siguiente paso. Ejemplo de uso:

```
cacheAside=newCacheAside(newRedisCache(redis));
$productos = cacheAside->obtener('productos:'.md5(jsonencode(filtros)), 300,
  fn () => /* consulta actual al ProductoRepository */ []);
```

3. Consistencia eventual

Dentro de MySQL, cada operación usa una transacción ACID (por ejemplo, CotizacionService::crear abre una transacción para la cabecera y los detalles). Entre almacenes se usa una saga coreografiada (ADR-006). El patrón Observer (sección 4) es el mecanismo que dispara cada paso:

Cambio de estado de la cotización	Reacción del observador de inventario

aprobada	Crea la reserva (activa)
rechazada	Cancela la reserva
convertida_a_venta	Completa la reserva

3.5 Aplicación de Patrones GoF y Análisis de Cohesión / Acoplamiento

Patrón	Clases	Ubicación	Estado
Repository	ClienteRepository, CotizacionRepository, ProductoRepository, UsuarioRepository	src/models/repositories/	Implementado y en uso
Strategy	MetodoPagoStrategy, EfectivoStrategy, ReferenciaBancariaStrategy, TarjetaStrategy, ProcesadorPago (contexto)	src/patterns/strategy/	Implementado; no conectado a un endpoint
Gateway	PasarelaPagoGateway, PasarelaPagoSimulada, RespuestaPasarela	src/patterns/gateway/	Implementado con pasarela simulada
Circuit Breaker	CircuitBreaker, CircuitoAbiertoException, PasarelaPagoConCircuitBreaker	src/patterns/resilience/ y src/patterns/gateway/	Implementado; el estado se guarda en la caché
Observer	CotizacionSujeto (SplSubject), ReservaInventarioObserver, BitacoraObserver (SplObserver)	src/patterns/observer/	Implementado; no conectado a CotizacionService

Cómo se relacionan:
Strategy elige cómo procesar cada valor del enum metodo_pago de la tabla pagos. Agregar un método nuevo es agregar una clase (escenario ES-01).
Gateway encapsula la pasarela externa; **Circuit Breaker** la protege. Si la pasarela falla, TarjetaStrategy deja el pago en pendiente en lugar de romper la solicitud.
Observer desacopla el cambio de estado de la cotización de sus consecuencias (reservas, bitácora).

5. Demostración

Desde la raíz del proyecto:
C:\xampp\php\php.exe tools/demo_patrones.php
Salida esperada (resumen): 1, 2 y 3 consultas a MySQL en la sección de caché; verificado, pendiente y verificado en los métodos de pago; el circuito pasa de cerrado a abierto tras 3 fallos y, con el circuito abierto, hay 0 llamadas reales a la pasarela; luego se recupera a cerrado; y el Observer crea, completa y cancela reservas según el estado.

6. Limitaciones

Strategy, Gateway, Circuit Breaker y Observer son módulos independientes, no conectados a los endpoints. Cache-Aside sí está conectado a ProductoService.

La pasarela de pago es simulada; MongoDB, Neo4j y Redis no están integrados en la aplicación.

Crear una cotización no invalida la caché; el stock mostrado puede estar atrasado hasta 5 minutos.

Los servicios existentes crean sus dependencias con new; para integrar estos módulos conviene aplicar inyección de dependencias por constructor.